

Agentic AI System for Enterprise Cloud Migration

Timothy Wong • Alessandro Abati

London Stock Exchange Group (LSEG)

7 Oct 2026

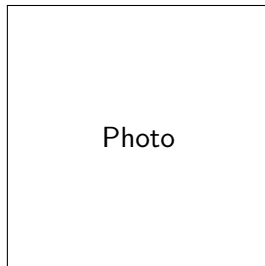


About the Speakers



Timothy Wong

Tim is an AI/ML engineer at LSEG with over ten years experience in enterprise data science applications. He hosted talks at EARL 2016, 17, 19 & 24.



Alessandro Abati

placeholder



LSEG

LSEG is a leading financial markets infrastructure and data provider operating connected businesses to serve customers across the entire financial markets value chain.

- ▶ **London Stock Exchange**
- ▶ **FTSE Russell:** Indices across asset classes, including FTSE 100, FTSE 250, Russell 2000, and ESG indices.
- ▶ **Data & Analytics:** Workspace platform, pricing, reference data.
- ▶ **Clearing:** LCH, multi-asset class clearing house.
- ▶ **Scale:** ~26 000 employees across many countries.

Agenda

1. **Business Requirements and Context**
2. **Agentic Technical Deep Dive**
 - 2.1 **Data Preprocessing**
 - 2.2 **Inside Agentic Retrieval**
 - 2.3 **Agentic Orchestration**
 - 2.4 **Agents' Memory**
 - 2.5 **Agents' Tools**
 - 2.6 **E2E Orchestration Phases**
3. **Application Overview**

- ▶ **Problem:** *placeholder* — what makes enterprise cloud migration slow, risky and manual.
- ▶ **Goal:** *placeholder* — automate discovery, analysis and recommendation with AI agents.
- ▶ **Stakeholders & scope:** *placeholder*.
- ▶ **Success criteria:** *placeholder*.

- ▶ **Ingestion:** *placeholder* — raw sources into Azure Blob / Table storage.
- ▶ **Knowledge sources & bases:** *placeholder* — Azure AI Search skillsets.
- ▶ **Chunking / enrichment:** *placeholder*.
- ▶ **Output:** *placeholder* — indexed data ready for agentic retrieval.

Inside Agentic Retrieval

- ▶ **What it is:** *placeholder* — Azure AI Search agentic retrieval vs. classic RAG.
- ▶ **Query planning:** *placeholder*.
- ▶ **Retrieval flow:** *placeholder*.
- ▶ **Grounding:** *placeholder*.

Agentic Orchestration — Overview

- ▶ **Framework:** *placeholder* — Microsoft Agent Framework.
- ▶ **Orchestration patterns used:** GroupChatManager, Concurrent, Writer/Reviewer.
- ▶ **When each pattern applies:** *placeholder*.

- ▶ **Pattern:** *placeholder* — manager coordinates a conversation between agents.
- ▶ **Turn / termination control:** *placeholder*.
- ▶ **Where used:** *placeholder*.

- ▶ **Pattern:** *placeholder* — fan-out agents run in parallel batches.
- ▶ **Aggregation:** *placeholder*.
- ▶ **Where used:** deep analysis batches, category-scoped UAQ agents.

- ▶ **Pattern:** *placeholder* — writer drafts, reviewer critiques, loop to convergence.
- ▶ **Quality gates:** *placeholder*.
- ▶ **Where used:** reporting phase.

- ▶ **What it is:** *placeholder* —
- ▶ **How it works:** *placeholder* — co-located `.agent.md` prompt, agent-owned memory, shared memory.
- ▶ **Where used:** *placeholder* — discovery

- ▶ **Agent anatomy:** factory `get_X_agent()`, co-located `.agent.md` prompt, agent-owned tools.
- ▶ **Tool catalogue:** `discovery_memory`, `file_system`, `search`, `grounding`, `ado_docs`, `sdp_docs`.
- ▶ **Shared middleware:** *placeholder* — Agent / Function / Chat / TokenUsage logging.

Agent-as-a-tool Pattern

- ▶ **Concept:** *placeholder* — exposing an agent as a callable tool for another agent.
- ▶ **Why:** *placeholder* — composition, encapsulation, reuse.
- ▶ **Example:** *placeholder*.

E2E Orchestration Phases — Overview

Discovery → Deep Analysis → Enterprise Context → UAQ →
Reporting → Cleanup

placeholder — how the phases hand off state end-to-end.

Discovery Phase

- ▶ **Goal:** *placeholder*.
- ▶ **Agents & tools:** *placeholder*.
- ▶ **Output:** *placeholder* — discovery memory.

Deep Analysis Phase

- ▶ **Goal:** *placeholder.*
- ▶ **Orchestration:** concurrent batches.
- ▶ **Output:** *placeholder.*

Enterprise Context Phase

- ▶ **Goal:** *placeholder* — enrich with LSEG/enterprise context.
- ▶ **Sources:** *placeholder*.
- ▶ **Output:** *placeholder*.

Unified Assessment Questionnaire (UAQ) Phase

- ▶ **Goal:** *placeholder* — Unified Assessment Questionnaire.
- ▶ **Orchestration:** category-scoped concurrent agents.
- ▶ **Output:** *placeholder*.

Reporting Phase

- ▶ **Goal:** *placeholder* — final recommendation report.
- ▶ **Orchestration:** writer/reviewer loop.
- ▶ **Output:** *placeholder*.

- ▶ **High-level architecture:** *placeholder* — diagram.
- ▶ **Stack:** Python 3.12, FastAPI, Poetry; React 19 frontend.
- ▶ **Data flow:** ingestion → preprocessing → agentic retrieval → orchestration.

- ▶ **Azure services:** *placeholder* — Blob / Table / Queue, AI Search, Key Vault, Foundry OpenAI.
- ▶ **Deployment:** *placeholder*.
- ▶ **Air-gapped considerations:** offline Mermaid CLI rendering, no remote fallback.

- ▶ **API:** *placeholder* — FastAPI endpoints.
- ▶ **Reliability:** `retry.py` `ErrorClass` (transient / deterministic / partial), exponential backoff + jitter.
- ▶ **Observability:** *placeholder* — OpenTelemetry token metrics.

- ▶ **Stack:** React 19.
- ▶ **Key views:** *placeholder.*
- ▶ **UX flow:** *placeholder.*

Thank you — Questions?